

README

This is the beginning of a server side web system for FPC. Although it is non-visual, it is geared towards use in Lazarus.

NOTE: the template support (fptemplate unit) was moved to fcl-base. See the fcl-base/texts/fptemplate.txt file.

Architecture:

httpprotocol:

Mostly standard HTTP header definitions, and some auxiliary routines.

httpdefs

contains the basic HTTP system definitions: header field names TCookie(s): collection with cookies in web request TUploadedFile(s): collection with uploaded files in request THTTPHeader:

Class describes HTTP Request/Response headers. it has properties for all possible HTTP headers, including cookies TRequest: Descendent of THTTPHeader, describes client request. Contains uploaded files. TResponse: describes the web server response. Includes headers and contents. TCustomSession: Base for all session components.

httproute

The old Delphi style routing worked with Datamodules only. The pattern was strictly /modulename/actionname or through query variables: ?module=xyz&Action=nmo

This old routing is still available by setting the LegacyRouting property of webhandler or webapplication (custweb) to true. (the new routing described below is then disabled)

The new routing is more flexible in 3 ways.

- It is no longer required to use datamodules, but this is still supported. There are now 4 methods that can be used to register a route:
 - Using a callback procedure TRouteCallback = Procedure(ARequest: TRequest; AResponse);
 - Using a callback event: TRouteEvent = Procedure(ARequest: TRequest; AResponse) of object;
 - Using an interface IRouteInterface = Interface ['{10115353-10BA-4B00-FDA5-80B69AC4CAD0}'] Procedure HandleRequest(ARequest : TRequest; AResponse : TResponse); end; Note that this is a CORBA interface, so no reference counting.
 - Using a router object: TRouteObject = Class(TObject,IRouteInterface) Public Procedure HandleRequest(ARequest : TRequest; AResponse : TResponse); virtual; abstract; end; TRouteObjectClass = Class of TRouteObject; The object

class needs to be registered. The router will instantiate the object and release it once the request was handled.

More methods can be added, if need be. All routes are registered using the `HTTPRouter.RegisterRoute` method. it is overloaded to accept any of the above parameters.

- The router can now match more complex, parametrized routes.

A route is the path part of an URL; query parameters are not examined.

`/path1/path2/path3/path4`

In these paths, parameters and wildcards are recognized: `:param` means that it will match any request with a single part in this location `*paramm` means that it will match any request with zero or more path parts in this location

examples:

`/path1`

`/REST/:Resource/:ID /REST/:Resource /*/something /*path/somethiingelse /*path`

The parameters will be added to `TRequest`, they are available in the (new) `RouteParams` array property of `TRequest`.

Paths are matched case sensitively by default, and the first matching pattern is used.

HTTP Modules are registered in the router using `classname/*` or `defaultmodulename/*`

- A set of methods can be added to the route registration (default is to accept all methods). The router will match the request method. If the method does not match, it will raise an exception which will result in a 405 HTTP error.

fphttp:

Basic web system components/classes

`TCustomHTTPModule`: Abstract `TDataModule` descendant, which processes a webrequest and prepares a response. `TModuleFactory/TModuleItem`: Module registration and creation system. `TCustomWebAction(s)`: Webactions. `RegisterHTTPModule()`: routine registers a module with the web system. `THTTPContentProducer` : abstract HTTP producer component.

The idea is that the URL relative to the server is taken and parsed as follows

<http://www.server.org/Path1/Path2>

Path1 determines which module in the system is created to handle the request. (the factory is queried) Path2 determines which web action inside the module is used to handle the request. (implemented in `TWebModule`, see `fpweb`)

websession

Implements basic session support.

TSessionHTTPModule:

TCustomHTTPModule descendent with session support. TIniWebSession: TCustomSession descendent which stores session variables in inifiles. TFPWebSession:

TIniWebSession descendent for use in fpweb. Uses cookies to store session info.

GetDefaultSession() : returns default session object.

fptemplate

Unit which implements template support.

TTemplateParser: Template parse object. Does the actual parsing and replacing. Delimiters are configurable. Standard variables can be specified, callbacks for unknown values can be set.

TFPCustomTemplate: TPersistent for use in components. Allows properties to configure the TTemplateParser. Supports streams, template and template files.

fpweb

Actual usable implementation of TCustomHTTPModule.

TFPWebAction(s): Web actions with template support. TCustomFPWebModule: Descends from TSessionHTTPModule, implements WebActions and has template support.

TFPWebModule: published TCustomFPWebModule properties for use in Lazarus.

fphtml

This creates web modules specialized in creating HTML content.

THTMLContentProducer: Descendent of THTTPContentProducer which produces HTML

THTMLCustomDatasetContentProducer: Descendent of THTTPContentProducer which produces HTML from datasets. THTMLDatasetContentProducer: Descendent of

THTMLCustomDatasetContentProducer which publishes some properties.

THTMLSelectProducer: Produces a combo box. THTMLDatasetSelectProducer Produces a combo box bases on values in a dataset. TCustomHTMLModule: TCustomHTTPModule descendent which produces HTML content only.

Note that classes for HTML document creation come from package fcl-xml.

fpdatasetform

This contains classes which allow to create complicated HTML/forms based on a TDataset.

THTMLDatasetFormProducer Creates an edit form for a TDataset record. Complicated table layouts are possible.

THTMLDatasetFormGridProducer Creates a grid with data from a TDataset Complicated table formatting is possible.

custcgi:

CGI application base class. It knows nothing of the fp HTTP module system.

TCustomCGIApplication : TCustomApplication descendent which handles a CGI request. No instance of this class is created, this is done in fpcgi.

TCGIRequest: TRequest descendent which retrieves content from the CGI environment.

TCGIResponse: TResponse descendent which returns content to the CGI environment.

fpcgi:

Standard CGI application instance.

TCGIApplication: TCustomCGIApplication descendent which uses the fpWeb system to create a TCustomHTTPModuleClass to handle the request.

It contains an Application instance which handles everything.

fpApache:

Apache Module support.

TCustomApacheApplication: Uses fpweb to create TCustomHTTPModuleClass instances to handle the request.

fphttpclient:

HTTP Protocol client component

TFPCustomHTTPClient: A component which can be used to communicate with a HTTP Server. It can execute HTTP GET and POST methods out of the box, but can be used to execute other HTTP methods as well. It works using the ssockets unit of FPC, so no third-party package to handle the socket communication is needed.

The class contains class methods, which allow to execute get/post methods in a really simple, one-command way.

A demo application for this class exists.

fphttpserver:

TFPCustomHttpServer: A Component which can be used to create a simple HTTP Server. All requests are routed through a OnRequest handler. The request and responses are modeled using the fpWeb TRequest and TResponse objects from httpdefs. The class itself does not serve files. The OnRequest handler must be implemented to actually serve files or respond to requests. It can work threaded or non-threaded. It works using the ssockets unit of FPC, so no third-party package to handle the socket communication is needed.

A demo application for this class exists.

custhttpapp:

Integration of TFPCustomHttpServer in a TWebApplication.

TFPHTTPServerHandler: A TWebHandler descendant which implements a stand-alone HTTP server. It uses the TFPCustomHttpServer component to implement the server.

TCustomHTTPApplication: A descendent of TCustomWebApplication which serves as an application object for stand-alone HTTP applications. It can be used as a parent component for standalone HTTP application objects.

fphttpApp:

THTTPApplication: A descendent of TCustomHTTPApplication. It does nothing except expose properties which exist in TCustomHTTPApplication. A global instance of this class is defined in fphttpApp. Use this unit and class if you want a simple standalone http server application for test purposes.

A demo application for this class exists.

fpwebfile:

Used to implement file serving.

TFPCustomFileModule: TFPCustomHTTPModule descendant which will serve files. Can be used as-is, but descendents can be made to implement e.g. logging, authorisation etc. must not be registered directly, register locations using the RegisterFileLocation call.

